

Evaluasi Kinerja Sistem Switching yang Mendukung SDN Untuk Infrastruktur IoT

Mohammad Naim Elham¹, Suriani Mohd Sam², Azizul Azizan³, Yusnaldi Md Yusof⁴, Norliza Mohamed⁵, Norulhusna Ahmad⁶
Razak Fakultas Teknologi dan Informatika, Universiti Teknologi Malaysia
Universiti Teknologi Malaysia,
Kuala Lumpur, Malaysia

Abstract- Revolusi Internet of Things (IoT) telah memberikan dampak besar pada infrastruktur jaringan dan teknologi informasi. Software Defined Networking (SDN) adalah cara untuk meningkatkan kelincuhan infrastruktur dan dengan demikian memfasilitasi desain, pengiriman, dan pengoperasian layanan jaringan yang dinamis dan terukur. Oleh karena itu, kelincuhan yang dihasilkan oleh pemrograman jaringan atau SDN sangat penting untuk mengatasi revolusi IoT yang mendorong infrastruktur ke batasnya dengan berbagai persyaratan yang harus dipenuhi. Node IoT biasanya bergantung pada lapisan jaringan untuk berkomunikasi satu sama lain. Namun, hampir tidak mungkin untuk membangun satu infrastruktur end-to-end yang dapat mengatasi seluruh kendala IoT. Tujuan dari proyek ini adalah untuk berbagi komponen dalam lapisan konvergen di mana Raspberry bertindak sebagai gateway IoT serta perangkat berkemampuan SDN yang dapat memenuhi berbagai persyaratan IoT. Raspberry Pi telah menjadi komoditas IoT yang umum selama beberapa tahun sekarang dan ditargetkan untuk didesain ulang untuk fungsionalitas jaringan tambahan. Versi terbaru #4 dari Raspberry Pi telah diinstal dengan Open vSwitch virtual yang sangat terkenal. Raspberry Pi berfungsi sebagai jalur data komunikasi jaringan di seluruh proyek dengan pengontrol SDN Ryu yang beroperasi di Raspberry Pi untuk memberikan kemampuan pemrograman. Sebuah sistem jaringan kabel, nirkabel dan SDN yang digabungkan kemudian dirancang untuk memvalidasi operasi Raspberry Pi. Setelah fungsionalitas berhasil, Raspberry Pi kemudian dievaluasi dalam hal parameter QoS (bandwidth, packet loss, delay dan jitter) untuk memastikan kesesuaian Raspberry Pi yang digunakan sebagai perangkat umum untuk operasi IoT dan SDN.

Karena lalu lintas jaringan berkembang pesat dengan munculnya komunikasi IoT, penerusan lalu lintas menjadi lebih mahal. Seperti yang dilaporkan oleh Business Insider, sebuah situs web berita keuangan dan bisnis Amerika, rata-rata untuk setiap proyek IoT, 35% dari biaya yang dikeluarkan terkait dengan perangkat keras seperti

Kata Kunci- *Internet of Things, Jaringan yang Ditentukan Perangkat Lunak, Peralihan, Kualitas Layanan.*

I. PENDAHULUAN

Internet of Things (IoT) memungkinkan pengumpulan dan pertukaran informasi secara real-time untuk menyediakan layanan yang heterogen bagi miliaran perangkat yang terhubung ke Internet. Pertumbuhan eksplosif Internet of Things menjanjikan banyak peluang baru yang menarik. Namun, hal ini juga menghadirkan masalah dan keterbatasan operasional dan kinerja tertentu. IoT akan memberikan dampak yang signifikan terhadap infrastruktur jaringan yang ada dan akan mendorong infrastruktur tersebut hingga ke batasnya, karena harus memenuhi berbagai macam persyaratan [1]. Oleh karena itu, infrastruktur IoT menuntut arsitektur dan teknologi baru yang dapat mendukung lalu lintas yang meningkat secara tak terduga untuk memfasilitasi sejumlah besar perangkat yang terhubung dengan karakteristik yang beragam dan kompleks.

yang dilaporkan di Business Insider, 2016. Menurut laporan lain dari Cisco, belanja perangkat keras konsumen dan bisnis dapat mencapai sekitar \$3 miliar pada tahun 2020 [2]. Oleh karena itu, diperlukan pendekatan inovatif dan cara yang hemat biaya untuk mengatasi tantangan manajemen lalu lintas IoT saat ini.

Perusahaan telekomunikasi Tiongkok, Huawei, memperkirakan bahwa industri teknologi informasi dan komunikasi (TIK) dapat menggunakan 20% listrik dunia dan melepaskan lebih dari 5% emisi karbon dunia pada tahun 2025. Pada tahun 2016, Lawrence Berkeley National Laboratory milik pemerintah Amerika Serikat memperkirakan bahwa pusat data Amerika - fasilitas tempat komputer menyimpan, memproses, dan berbagi informasi - mungkin membutuhkan 73 miliar kWh energi pada tahun 2020. Dunia menghasilkan lebih banyak lagi limbah listrik dan elektronik. Jumlah komputer, telepon, televisi, dan peralatan yang dibuang meningkat dua kali lipat antara tahun 2009 dan 2014, menjadi 42 juta ton per tahun secara global dan tren ini akan terus meningkat menurut statistik di bawah ini [3].

Untuk mengatasi masalah ini, jaringan yang ditentukan oleh perangkat lunak (SDN) telah muncul akhir-akhir ini, untuk mengatasi tantangan dan persyaratan IoT yang disebutkan di atas dalam upaya memisahkan bidang kontrol jaringan dari bidang data. Bidang data terdiri dari switch dan router yang memfasilitasi pertukaran paket dalam jaringan, sedangkan antarmuka 'bidang kontrol' berfungsi sebagai interkoneksi antara pengontrol dan switch di bidang data dan mendefinisikan interaksi di antara keduanya. SDN menyajikan aplikasi di lapisan atas dengan abstraksi dari jaringan yang mendasarinya. SDN menggunakan 'control plane' sebagai elemen perangkat lunak, yang berada di server, namun menggunakan 'data plane' pada perangkat jaringan. Karena fleksibilitas dalam mengendalikan bidang penerusan ini, memungkinkan peralatan jaringan untuk memperoleh fungsi baru dan tidak harus diganti ketika persyaratan baru diperlukan. SDN telah berkembang secara substansial di industri selama beberapa tahun terakhir. Vendor pertama yang menyediakan fitur OpenFlow dalam produk mereka termasuk HP, NEC, dan Pronto [4]; sejak saat itu, daftar tersebut telah berkembang secara dramatis.

Meskipun SDN telah sukses, sakelar yang kompatibel dengan OpenFlow masih mahal. Anda tidak akan menyangka bahwa investasi sakelar berkemampuan SDN sebesar \$1.000 untuk perangkat rumahan pengguna akhir [5]. Akibatnya, perangkat SDN yang fleksibel dan dinamis masih hanya digunakan di perusahaan-perusahaan dengan anggaran yang memadai. Karena kendala ini, ada peningkatan permintaan untuk alternatif SDN berbiaya rendah yang dapat mengarahkan SDN ke implementasi skala yang lebih kecil seperti rumah dan lingkungan perusahaan beranggaran rendah. Gagasan tentang perangkat keras yang benar-benar murah (yaitu perangkat yang harganya kurang dari \$100) muncul untuk membuat peralatan SDN dapat diakses. Peralatan berbiaya rendah memang tidak sempurna, biaya rendah menyiratkan keterbatasan dan masalah.

II. JARINGAN YANG DITENTUKAN PERANGKAT LUNAK

Lapisan yang mendasari termasuk lapisan kontrol dan bidang kontrol. Lapisan ini mengatur semua aktivitas dalam

Software Defined Networking (SDN) adalah pendekatan baru untuk jaringan yang dapat diprogram. Pendekatan ini mengubah proses desain, implementasi, dan manajemen jaringan. Pendekatan ini memiliki dua ciri khas. Pertama, memperkenalkan lapisan abstraksi yang memisahkan bidang kontrol dari bidang data. Pemisahan bidang kontrol dari bidang data membuat manajemen jaringan menjadi lebih mudah. Dengan cara ini, perangkat lunak komputer memiliki gambaran umum tentang seluruh jaringan, yang dinamakan sebagai 'pengontrol' dan tanggung jawabnya adalah untuk mengetahui kondisi jaringan dan mengambil keputusan. Perangkat keras hanya mentransmisikan paket ke tujuannya, seperti yang diinstruksikan oleh pengontrol. Secara umum, instruksi adalah sekumpulan aturan penanganan paket [6]. Kedua, konsolidasi bidang kontrol membentuk manajemen jaringan yang terpusat di mana hanya satu program yang menangani berbagai elemen yang berbeda dari paket data (yaitu switch, router ...) sebagai lawan dari paradigma tradisional di mana manajemen didistribusikan dan perangkat yang berdiri sendiri-sendiri harus dikelola secara individual.

Penerusan lalu lintas SDN didasarkan pada aturan 'match-action' sederhana di bidang data yang secara signifikan meningkatkan kecepatan pengambilan keputusan. Menurut laporan dari Google [7], SDN telah membantu meningkatkan pemanfaatan WAN hingga mendekati 100%. Evolusi SDN telah terjadi dalam 3 tahap yang mencakup hampir 20 tahun penelitian dan kontribusi akademis [8]. Dalam arsitektur SDN, pengontrol tidak harus tunggal. Beberapa pengontrol dapat berjalan secara bersamaan untuk mempertahankan ukuran infrastruktur yang lebih besar karena perusahaan besar atau jaringan IoT akan membutuhkan banyak pengontrol.

OpenFlow, yang distandarisasi oleh Open Networking Foundation (ONF), adalah antarmuka jalur selatan yang paling umum. OpenFlow adalah protokol yang menjelaskan interaksi dengan sakelar yang sesuai dengan OpenFlow antara satu atau lebih server kontrol. Pengontrol OpenFlow memasang entri tabel aliran di sakelar sehingga menurut entri ini, sakelar ini dapat meneruskan lalu lintas. Sakelar OpenFlow karena itu bergantung pada konfigurasi pengontrol. Aliran diklasifikasikan berdasarkan bidang yang cocok yang mirip dengan ACL dan mungkin berisi wildcard.

Arsitektur SoftRouter juga mendefinisikan kontrol dan fungsionalitas bidang data yang terpisah. Hal ini memungkinkan pengikatan dinamis antara elemen kontrol dan elemen bidang data, menyediakan Masa Depan Internet yang dinamis. ForCES dan SoftRouter mirip dengan OpenFlow dan dapat memainkan peran antarmuka selatan. Juga dibahas teknologi jaringan lainnya, serta kemungkinan antarmuka southbound IETF. Sebagai contoh, Path Computation Element (PCE) dan Locator / ID Separation Protocol (LISP) adalah kandidat interface southbound [9].

Lapisan atas terdiri dari lapisan aplikasi dan aplikasi jaringan yang memperkenalkan karakteristik jaringan yang unik, memungkinkan lapisan kontrol untuk mengelola tabel aliran dan skema penerusan [10]. Lapisan ini memperoleh dari pengontrol di bawahnya pandangan abstrak dan global dari seluruh jaringan, yang pada gilirannya mengontrol semua perangkat jaringan di lapisan infrastruktur. Antarmuka northbound adalah antarmuka antara lapisan aplikasi dan lapisan kontrol seperti yang ditunjukkan pada Gbr. 1.

lapisan yang mendasarinya. Dalam situasi ini, controller memainkan peran penting antara lapisan aplikasi dan lapisan infrastruktur [11]. Dalam hal penanganan dan pemrograman sakelar, Controller dianggap memiliki semua otak. Ini berinteraksi melalui antarmuka selatan dengan switch yang terletak di lapisan bawah yaitu lapisan infrastruktur. Lapisan bawah terdiri dari peralatan jaringan, yang sebagian besar adalah switch

/ router yang tidak memiliki kemampuan untuk mengambil tindakan di dalamnya dan memungkinkan pengontrol untuk mengambil alih tabel aliran. Ini adalah tabel aliran dalam switch. Lapisan ini memiliki perangkat keras yang telah mengimplementasikan OpenFlow termasuk switch / router [12].

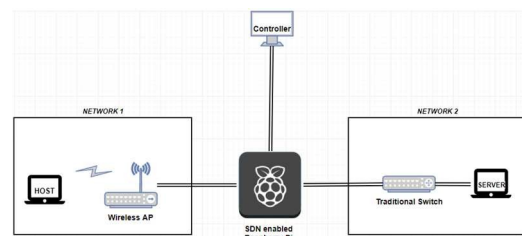
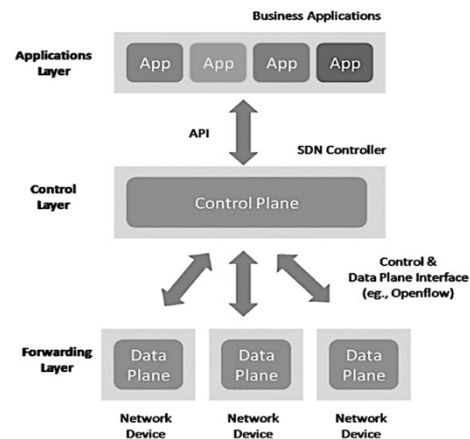
Gbr. 1. Arsitektur SDN

III. ARSITEKTUR SISTEM

Arsitektur sistem sering kali terdiri dari perangkat keras sistem dan subsistem yang terintegrasi yang akan bekerja satu sama lain untuk menjalankan jaringan secara keseluruhan. Sistem yang dikembangkan untuk pekerjaan ini terdiri dari 3 sub-unit. Unit jaringan nirkabel, unit jaringan kabel dan terutama unit yang mendukung SDN. Masing-masing unit ini akan dibahas lebih lanjut pada bagian selanjutnya. Pengontrol akan diposisikan pada laptop untuk tujuan pelacakan dan kenyamanan konfigurasi, dan Ryu hanyalah salah satu perangkat lunak pengontrol yang dipilih untuk digunakan. Idealnya, switch perangkat lunak yang dapat berjalan di Raspberry Pi, memiliki fungsi yang memungkinkan Anda untuk mengontrol parameter QoS pada switch, serta mudah digunakan akan digunakan. Jika pengontrol akan dikonfigurasi, akan lebih cocok jika ditulis dalam bahasa pemrograman yang mirip dengan python untuk mengurangi kesalahan.

Gbr. 2. Arsitektur sistem

Pada Gbr. 2, arsitektur terdiri dari dua jenis jaringan yang berbeda. *Jaringan 2* terdiri dari jaringan LAN biasa di mana host terhubung ke sakelar tradisional di rak. Sambungannya jelas berkabel dan menggunakan kabel tembaga. Jaringan 1 terdiri dari sebuah AP dan laptop yang bertindak sebagai server. Jaringan ini menyerupai jaringan WLAN pada umumnya di mana koneksinya melalui gelombang radio dan Wi-Fi. Jaringan 2 terdiri dari AP dan laptop yang bertindak sebagai server.



Jaringan SDN terdiri dari Raspberry Pi yang memiliki Open vSwitch yang telah diinstal dan dikonfigurasi pada jembatannya. Raspberry Pi akan bertindak sebagai switch berkemampuan SDN yang dapat mendukung perangkat bidang data SDN yang lengkap. Namun, untuk control plane laptop lain yang terhubung langsung ke port Ethernet asli Raspberry Pi yang dapat membantu Raspberry Pi dalam mengambil keputusan.

Dari segi persyaratan perangkat keras sistem, unit jaringan nirkabel pada dasarnya adalah jaringan WLAN. Hal ini memungkinkan dua perangkat atau lebih untuk berkomunikasi menggunakan gelombang radio nirkabel untuk membentuk LAN dalam area yang terbatas. Dengan demikian, LAN adalah jaringan yang lengkap di mana anggota LAN dapat dengan mudah terhubung satu sama lain tanpa layanan perutean sekunder. Setiap perangkat yang dapat diakses dalam jaringan media nirkabel disebut stasiun (STA). Dua jenis STA adalah titik akses nirkabel (WAP) dan klien. Unit jaringan nirkabel bertanggung jawab untuk merepresentasikan jaringan dunia nyata. Proyek ini menggunakan jaringan ini dan khususnya node host untuk menghasilkan data lalu lintas simulasi dengan parameter yang berbeda untuk melewati jaringan dan mencapai node server Network 2. Unit jaringan kabel menggunakan sakelar jaringan tunggal untuk membuat dan mengelola semua perangkat yang terhubung dalam jaringannya sendiri seperti yang ditunjukkan pada Gbr. 3. Sakelar berfungsi sebagai titik penghubung yang penting dan memungkinkan komunikasi antara perangkat seperti komputer satu sama lain. Pada dasarnya ini adalah jaringannya sendiri. Namun, ia juga harus dapat berkomunikasi dengan jaringan lain seperti *Jaringan 1* di mana host berada. Sebuah komputer ditunjuk sebagai titik pusat LAN ini. Komputer ini bertindak sebagai server, mendengarkan node host *Jaringan 1* dan merespons paket.

Desain sakelar SDN untuk komunikasi IoT memerlukan penggunaan komponen yang ringan dan murah. Oleh karena itu, Raspberry Pi dapat digunakan sebagai komputer papan tunggal berdaya rendah, ringkas, dan ekonomis untuk menjalankan proses penerusan paket yang diterima dari satu port ke port lain yang dituju agar paket tersebut mencapai tujuan yang diinginkan. Ketika dikombinasikan dengan perangkat lunak switching open source (OpenFlow) [13], hasil akhirnya adalah solusi IoT yang hemat biaya. Raspberry Pi dioperasikan oleh sistem operasi Raspbian menggunakan versi terbaru (per Maret 2020). Kernel perangkat lunak yang menjalankan fungsionalitas jaringan disediakan oleh Open vSwitch. Bagian jaringan ini membutuhkan pengontrol untuk mengontrol jalur data Raspberry Pi. Pengontrol Ryu digunakan untuk mengelola vSwitch pada mesin jarak jauh.

Sistem yang dirancang dalam hal kebutuhan perangkat lunak cukup beragam. Sangat penting bagi komponen perangkat lunak untuk dikelola dengan baik dan agar seluruh sistem bekerja secara efisien. Dengan demikian, langkah selanjutnya untuk mengaktifkan Raspberry sebagai saklar OpenFlow adalah perlunya menambahkan saklar perangkat lunak ke Raspberry Pi. Software switch yang digunakan dalam proyek ini adalah Open vSwitch. Skema bagaimana software switch OpenFlow dibangun pada Raspberry dijelaskan pada Gambar 4 di bawah ini.

Raspberry Pi versi 4 memiliki 4 port USB. Tiga dari 4 port USB tersebut dilengkapi dengan dongle USB-to-Ethernet. Dengan demikian, jumlah port Ethernet untuk Raspberry Pi bertambah dari yang semula hanya satu port

menjadi empat port. Adaptor USB-to-Ethernet bekerja secara langsung tanpa harus menyiapkan driver atau aplikasi tambahan pada sistem operasi Raspbian. Namun, sebuah aturan dapat dibuat di file `/etc/udev/rules.d/70-persistent-net.rules` untuk pemetaan antarmuka yang konsisten untuk masing-masing nama perangkat.

Namun demikian, agar port-port tersebut dapat berkomunikasi, kita perlu membuat jembatan jaringan di antara mereka sendiri. Jika tidak, port USB tidak dirancang untuk berkomunikasi satu sama lain secara default melainkan hanya ke sistem RPi. Untuk membuat jembatan di dalam Raspberry Pi, kita akan menggunakan perangkat lunak sumber terbuka multi-layer yaitu Open vSwitch. Open vSwitch memungkinkan pengembang jaringan untuk memperkenalkan platform switch berkualitas produksi pada sistem berbasis Linux. Fitur-fitur OVS termasuk antarmuka manajemen standar dan menyediakan fungsi ekspansi dan transmisi terprogram. Win32DiskImager adalah aplikasi utilitas Windows sederhana yang memungkinkan Anda untuk menuliskan gambar ke media (Write) atau akuisisi (Read). Sistem kemudian dapat dipersiapkan untuk melihat kartu SD sebagai kartu yang dapat di-boot dan untuk mem-boot sistem operasi.

Gbr. 3. Unit jaringan nirkabel & unit jaringan berkabel

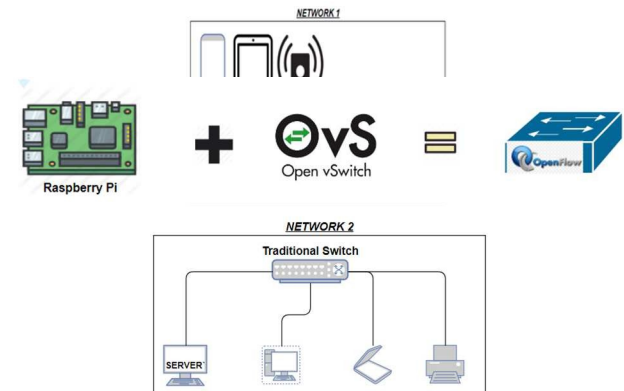
Gbr. 4. Kebutuhan perangkat lunak untuk RPi

IV. KONFIGURASI SISTEM

Konfigurasi sistem adalah istilah dalam rekayasa sistem yang mendefinisikan perangkat keras komputer, proses, serta berbagai perangkat yang terdiri dari keseluruhan sistem dan batas-batasnya. Konfigurasi sistem juga mengacu pada pengaturan atau pengaturan perangkat keras-perangkat lunak dan bagaimana setiap perangkat dan perangkat lunak atau proses berinteraksi satu sama lain berdasarkan file pengaturan sistem yang dibuat secara otomatis oleh sistem atau ditentukan oleh pengguna.

Access Point dapat dikonfigurasi dalam hal SSID, saluran, daya transmisi, pemfilteran MAC, fungsionalitas NAT dan masih banyak lagi. Satu-satunya konfigurasi yang sangat penting untuk proyek ini adalah penetapan IP atau server DHCP. Seperti yang disebutkan dalam arsitektur jaringan proyek, jaringan nirkabel memiliki subnet 192.168.10.0. Dengan demikian, jaringan ini akan mendistribusikan alamat IP dari kelompok ini.

Sakelar mengacu pada unit perangkat keras yang menghubungkan jaringan. Dengan mengakomodasi beberapa perangkat, sakelar menghubungkan seluruh Jaringan Area Lokal (LAN). Karena sakelar tersebut menghubungkan berbagai perangkat jaringan, ukurannya dapat berbeda antara 5 dan 48 port. Kedua port ini harus menangani kabel yang menggunakan jaringan yang sama dari berbagai perangkat [19]. Sakelar jaringan terkelola ini menyediakan konfigurasi



tunjangan. Dengan demikian, alamat IP statis telah ditetapkan pada salah satu port Fast Ethernet dan juga nama host telah diubah untuk mencerminkan proyek.

Setelah memastikan bahwa Raspberry Pi telah diperbarui ke distribusi terbarunya dan menginstal Open vSwitch dengan sukses seperti yang telah dibahas sebelumnya, langkah logis berikutnya adalah mengkonfigurasi Open vSwitch untuk beroperasi sebagai sakelar perangkat lunak virtual berbasis Raspberry Pi dan memfasilitasi jalur data antara berbagai amandemen koneksi USB-ke-Ethernet Raspberry Pi. Perintah *ovs-vsctl* pertama adalah yang paling penting untuk mengonfigurasi dan mengontrol sakelar perangkat lunak virtual. Idenya adalah untuk menginisialisasi Open vSwitch secara umum menyediakan jembatan antara setiap antarmuka perangkat yang terlibat dalam jaringan OpenFlow. Tindakan jembatan tersebut kemudian akan ditentukan oleh aturan aliran. Pertama, jembatan virtual harus dibangun dan antarmuka jembatan virtual dengan nama yang sama akan dibuat secara otomatis. Jembatan ini dapat memiliki lebih banyak antarmuka, jika perlu. Perintah *ovs-vsctl show* akan mencetak gambaran umum singkat tentang isi database Open vSwitch seperti yang ditunjukkan pada Gbr. 5.

Sakelar OpenFlow dapat dihubungkan ke pengontrol melalui saluran TCP. Hal ini juga diperlukan untuk menyediakan konektivitas IP antara sakelar dan pengontrol [14]. Dalam pekerjaan ini, koneksi yang digunakan digunakan melalui antarmuka jembatan, yang juga digunakan untuk perutean lalu lintas, dan dengan demikian ditugaskan ke jembatan. Dalam hal ini, alamat IP tidak diatur ke antarmuka jaringan mana pun, tetapi pada bridge, yang juga ditemukan dalam daftar antarmuka jaringan, misalnya dari perintah *ifconfig*. Pengontrol akan dimulai dari direktori dengan kode sumbernya menggunakan perintah *./bin/ryu-manager -verbose* seperti yang ditunjukkan pada Gbr. 6.

Sakelar OpenFlow dapat dihubungkan ke pengontrol melalui saluran TCP. Hal ini juga diperlukan untuk menyediakan konektivitas IP antara sakelar dan pengontrol [14]. Dalam pekerjaan ini, koneksi yang digunakan digunakan melalui antarmuka jembatan, yang juga digunakan untuk perutean lalu lintas, dan dengan demikian ditugaskan ke jembatan. Dalam hal ini, alamat IP tidak diatur ke antarmuka jaringan mana pun, tetapi pada bridge, yang juga ditemukan dalam daftar antarmuka jaringan, misalnya dari perintah *ifconfig*. Pengontrol akan dimulai dari direktori dengan kode sumbernya menggunakan perintah *./bin/ryu-manager -verbose* seperti yang ditunjukkan pada Gbr. 7.

Untuk konfigurasi host dan server, dua node yang disebutkan adalah 2 laptop yang di-boot dari USB dengan Ubuntu Linux 18.04 LTS. Node klien dikonfigurasi untuk menerima alamat IP dinamis yang diberikan oleh jalur akses nirkabel

```
root@raspberrypi:/home/pi# ovs-vsctl show
2204c668-6879-4dc9-9bdf-ee5c929bc23a
Bridge "br0"
  Controller "tcp:192.168.0.100"
  fail_mode: secure
  Port "eth1"
    Interface "eth1"
  Port "eth0"
    Interface "eth0"
  Port "br0"
    Interface "br0"
    type: internal
  Port "eth2"
    Interface "eth2"
ovs_version: "2.10.1"
```

Gbr. 5. Perintah tampilkan OVS-vsctl

Server DHCP. Dengan cara ini, server akan menerima salah satu alamat IP yang ditetapkan dari kumpulan subnet 192.168.10.0/24. Node server masing-masing diberi alamat IP statis

Gbr. 6. Proses inialisasi Ryu

Perintah untuk menetapkan alamat IP ke mesin Linux menggunakan perintah *ifconfig*. Subnet yang digunakan untuk jaringan ini adalah subnet mask kelas C default 255.255.255.0 atau /24. Perintah *up* akan menyalakan eth1 atau port apa pun yang dipilih seperti yang ditunjukkan pada Gbr. 7.

Gbr. 7. Konfigurasi IP server

V. PENGATURAN EKSPERIMENTAL

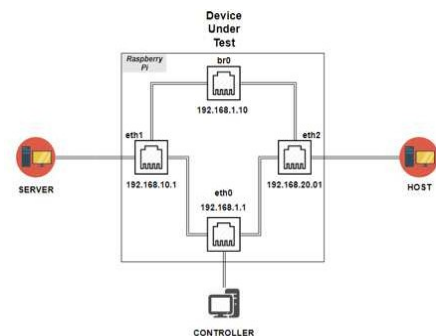
Meskipun detail teknis akan bervariasi dengan pengujian yang berbeda, desain keseluruhan eksperimen akan mengikuti arsitektur SDN umum yang ditunjukkan pada Gbr. 8. Pengontrol yang ringan dan terdesentralisasi, Ryu, akan berjalan di atas bidang kontrol. Pada lapisan aplikasi, iPerf akan digunakan untuk menghasilkan dan memantau dampak dari parameter QoS. Gambar 9. Dua komputer yang menjalankan Desktop Ubuntu yang baru 18.045 dengan Kernel Linux 4.10 akan digunakan untuk pengaturan eksperimen kami. Kasus uji menggunakan topologi Leaf-Spine dalam jaringan untuk perbandingan yang memiliki setidaknya 1 node yang mendukung SDN. Node yang mendukung SDN harus selalu terhubung ke HOST penghasil lalu lintas evaluasi dan SERVER penerima lalu lintas.

Gbr. 8. Diagram perangkat yang sedang diuji

Sistem yang dikembangkan dalam proyek ini diwujudkan dengan perangkat keras yang nyata di aula apartemen saya. Tiga laptop dipasang dengan sistem operasi Ubuntu untuk menjalankan *Iperf* dan *net-tools* yang terpasang di dalamnya untuk melakukan validasi dan evaluasi proyek ini. Dua dari laptop berfungsi sebagai klien dan server Iperf untuk meniru dan menghitung trafik yang dihasilkan. Laptop ketiga terhubung langsung ke jaringan

```
#/bin/ryu-manager -verbose
loading app ryu.controller.ofp_handler
Instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK ofp_event
loading app ryu.controller.ofp_handler
Instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK ofp_event
CONSUMES EventOFPHello
CONSUMES EventOFPSwitchFeatures
CONSUMES EventOFPEchoRequest
CONSUMES EventOFPErrormsg
CONSUMES EventOFPPortStatus
CONSUMES EventOFPPortDescStatsReply
-----Continued in Appendix-----
```

```
root@projectpc-VirtualBox: /home/projectpc
root@projectpc-VirtualBox:~# ifconfig enp0s3 192.168.20.10 255.255.255.0
root@projectpc-VirtualBox:~# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 255.255.255.0 netmask 255.255.255.0 broadcast 0.0.0.0
    inet6 fe80::1e46:197d:2aa0:1f35 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:e1:71:5c txqueuelen 1000 (Ethernet)
    RX packets 2123 bytes 2814913 (2.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1036 bytes 98119 (98.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```



Raspberry Pi, dipasang Ryu sebagai pengendali SDN dan berjalan sebagai sakelar pembelajaran sederhana untuk memfasilitasi layanan peralihan jaringan seperti yang ditunjukkan pada Gbr. 9. Titik akses nirkabel disiapkan dengan server DHCP dan terhubung secara nirkabel ke satu laptop dan juga terhubung dengan kabel ke salah satu port Ethernet Raspberry Pi. Sakelar tradisional yang dikelola juga dikonfigurasi dengan alamat IP statis dan terhubung ke Port Ethernet kedua Raspberry Pi. Raspberry Pi berfungsi sebagai media fisik untuk menghubungkan dua jaringan kabel dengan berbagai subnet IP. Dengan demikian, dianggap perlu untuk mengevaluasi jalur data yang disediakan Raspberry Pi dan dengan demikian, tahap evaluasi proyek ini dilakukan untuk mengevaluasi efisiensi Raspberry Pi di lingkungan dunia nyata.

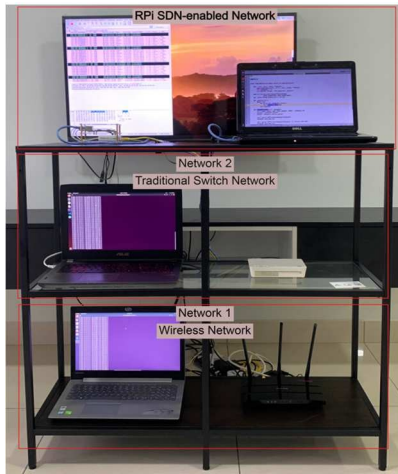
Aliran paket antara dua node jaringan disebut aliran [15]. Aliran ini harus mengambil rute yang sama melalui jaringan yang berorientasi pada koneksi, tetapi paket akan mengambil jalur yang berbeda dalam jaringan tanpa koneksi. Kemungkinan rute memiliki atribut yang berbeda dengan jaringan tanpa koneksi [16]. Empat fitur koneksi jaringan utama adalah throughput, packet loss, delay dan jitter.

Throughput adalah ukuran jumlah data yang dapat dikirim melalui sambungan dalam waktu tertentu. Throughput diukur dalam bit per detik. Ekspresi yang umum digunakan adalah dalam kilobit (10^3 bit), megabit (10^6 bit) atau gigabit (10^9 bit) per detik. Perbedaan antara bandwidth dan transmisi adalah bahwa pengukuran transmisi dapat dipengaruhi oleh biaya overhead yang besar yang tidak termasuk dalam perhitungan bandwidth. Dua faktor yang mempengaruhi throughput adalah jumlah data yang ditransfer, dan waktu yang dibutuhkan untuk mentransfer data tersebut. Menentukan throughput dapat dilakukan dengan mengukur waktu yang dibutuhkan untuk mentransfer sejumlah data yang telah ditentukan. Iperf adalah alat pengukuran dinamis dengan jaringan IP, di mana tujuan utamanya adalah untuk menilai throughput jaringan IP maksimum yang dapat dijangkau. Alat ini juga dapat digunakan untuk metrik lain seperti jitter, dan kehilangan paket.

Packet loss didefinisikan sebagai persentase paket yang benar-benar sampai dari sumber ke penerima, terhadap semua paket yang dikirim dalam satuan waktu tertentu. Packet loss dapat dievaluasi dengan perintah *ping* atau evaluasi throughput Iperf UDP. Ping adalah sebuah prompt perintah yang digunakan untuk menguji kemampuan komputer untuk mencapai komputer tujuan tertentu. Perintah ping bekerja dengan mengirimkan pesan echo request ke komputer tujuan melalui Internet Control Message Protocol (ICMP) [17]. Dua informasi utama yang disediakan oleh perintah ping meliputi berapa banyak respons yang dikembalikan dan berapa lama waktu yang dibutuhkan untuk mengembalikannya.

Penundaan adalah waktu yang diperlukan untuk mengirim paket atau frame dari node sumber ke node tujuan. Penundaan adalah hasil dari tiga penundaan, yang disebut penundaan transmisi, penundaan propagasi dan penundaan antrian. Penundaan dinyatakan dalam waktu, dan karena penundaan biasanya cukup kecil, maka dinyatakan dalam milidetik. Penundaan pulang pergi adalah yang paling berguna untuk sebagian besar aplikasi karena ada interaksi antara dua host yang berkomunikasi. Ping dan Iperf keduanya dapat digunakan untuk mengevaluasi RTT sebuah jaringan. Jitter adalah variasi waktu kedatangan paket yang

berurutan dari sumber ke tujuan dan ditentukan oleh perbedaan yang dialami oleh paket berikutnya, RTTI dan RTTI+1 [18]. Semua alat yang dapat digunakan untuk mengukur RTT juga dapat digunakan untuk memeriksa Jitter termasuk ping, trace route dan Iperf. Alat-alat tersebut secara khusus termasuk Iperf ketika dilakukan dengan paket UDP.



Gbr. 9. Penyiapan eksperimen di laboratorium

Tahap pertama mengumpulkan data mentah dari jaringan atau komputer. Hal ini bisa dilakukan dengan pengukuran aktif, yaitu alat yang menghasilkan lalu lintas di jaringan untuk melakukan pengukuran. Program Iperf dapat mengeluarkan hasilnya ke dalam file teks untuk pemeriksaan lebih lanjut. File-file ini biasanya disimpan di direktori `C:\iperf3\text.txt` setelah program tersebut mengumpulkan semua informasi setelah jangka waktu tertentu.

VI. HASIL DAN ANALISIS

Beberapa kasus uji diimplementasikan. Pada kasus uji pertama, penundaan sambungan jaringan akan diukur dan berdasarkan pengukuran tersebut, kehilangan paket akan ditentukan. Penundaan dapat dievaluasi sebagai jumlah waktu yang diperlukan untuk mengirim paket dari sebuah host, sampai paket tersebut diterima di tempat tujuan. Tetapi karena hal ini membutuhkan sinkronisasi jam yang sempurna, metode alternatif lebih sering digunakan. Metode yang terakhir ini menghitung penundaan dalam konteks waktu pulang pergi. Datagram ICMP maksimum tergantung pada jenis OS yang berbeda. Pada jaringan Ethernet, MTU maksimum adalah 1500. Ketika sebuah host mentransmisikan datagram IP yang lebih besar dari ukuran MTU, Raspberry Pi memecah datagram menjadi potongan-potongan yang lebih kecil. Proses ini dapat menyebabkan penundaan dan kehilangan paket. Itulah alasan mengapa kasus uji coba mempertimbangkan untuk mengganti ukuran paket selama pengujian untuk lebih mencerminkan kemampuan kinerja Raspberry Pi yang bersangkutan.

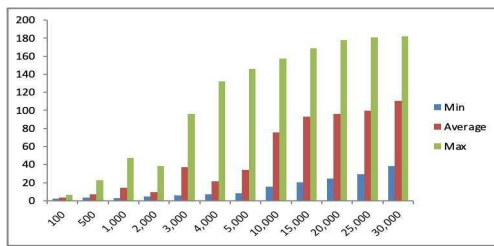
Probabilitas penurunan paket biasanya meningkat dengan memaksimalkan ukuran paket. Karena lebih banyak waktu yang dibutuhkan untuk mengirimkan paket besar dari sumber ke tujuan, kemungkinan kemacetan serta alasan lain untuk paket drop, misalnya batas waktu pengiriman ulang TCP meningkat. Penundaan paket juga meningkat dengan meningkatnya ukuran paket. Paket yang lebih besar membutuhkan lebih banyak waktu untuk mencapai tujuannya dibandingkan dengan paket yang lebih kecil; tidak hanya itu, tampaknya karena lebih banyak paket yang jatuh, lebih banyak pengiriman ulang paket yang diperlukan; oleh karena itu, penundaan umumnya meningkat. Dalam aplikasi IoT, ukuran paket umumnya kecil seperti kesehatan dan perangkat yang dapat dikenakan, bantuan pintar, otomatisasi rumah, dan peralatan kerja. Namun, ada beberapa aplikasi tertentu

seperti konsol game, kamera pintar, dan TV pintar yang membutuhkan kemampuan menangani ukuran paket data yang besar. Gbr. 10 menunjukkan tren bagaimana delay dan packet loss terkait dengan ukuran paket sesuai dengan kasus pengujian.

Pada kasus pengujian kedua, ini mengukur throughput koneksi dua arah dan satu arah yang tersedia. Hasil

hubungan antara dua node harus dibandingkan dan dianalisis dengan menggunakan alat pengukuran aktif Iperf.

kecepatan data yang tinggi untuk node



Gbr. 10. Tren Kasus Uji #1

Kasus uji akan memberikan informasi yang cukup untuk melihat pola jaringan, dan menentukan apakah node berhasil menggunakan total bandwidth yang tersedia. Jika kecepatan koneksi tidak sesuai dengan yang diantisipasi, kemacetan dapat diambil dan dihilangkan. Setiap kali throughput dua arah dicatat, jumlah total gabungan data yang masuk ke kedua arah sebenarnya dihitung, sehingga throughput dua arah biasanya dua kali lebih besar daripada satu arah, tetapi dalam praktiknya, setiap kali data ditransmisikan di kedua arah secara bersamaan, biasanya terjadi penurunan kinerja pada Raspberry Pi.

TABEL I. HASIL KASUS UJI COBA #2

No	Data Transfer (Mbytes)	Maximum Throughput (Mbits/s) (Client-Server)	Maximum Throughput (Mbits/s) (Server- Client)	Specified Theoretical Rate (Mbits/s)
1	112 Mb	93.5 Mb/s	-	100 Mb/s
2	113 Mb	-	94.9 Mb/s	100 Mb/s
3	118 Mb	98.8 Mb/s (Unidirectional)		100 Mb/s

Hasilnya ditunjukkan pada Tabel I. Pengujian pertama berhasil mencapai 93,5 Mb/s dari Host ke server dan 94,9 Mb/s dari server ke klien. Pengujian satu arah mencapai 98,8 Mb/s yang sangat dekat dengan throughput maksimum secara teoritis. Pada saat yang sama, beban CPU berada pada kapasitas maksimum. Throughput teoritis seharusnya 100 Mb/s. Tidak mungkin untuk keluar dari koneksi apa pun dengan pemanfaatan 100 persen. Biasanya, penggunaan 95% adalah batas yang dapat dicapai di dunia nyata. Jika lebih dari itu, jaringan akan mulai memenuhi koneksi dan kehilangan paket.

Kasus uji ketiga mengukur throughput yang tersedia dengan jumlah data dan jenis paket UDP yang tetap. Menguji performa UDP di Iperf berbeda dengan menguji performa TCP. Beberapa nilai parameter seperti throughput dan kemudian mengirim data selama periode waktu tertentu untuk mengamati delay/jitter dan juga persentase datagram yang hilang. Pengujian UDP tidak hanya memberikan informasi berharga tentang jitter tetapi juga packet loss. Jitter adalah variasi dari latency, dan tidak tergantung pada latency itu sendiri. Jitter yang tinggi dapat menimbulkan masalah serius dan kadang-kadang bahkan merusak panggilan VoIP. Tes UDP juga dapat mengukur kehilangan paket jaringan. Sambungan yang berkualitas baik tidak boleh kehilangan lebih dari 1% dari total paket.

Tabel II menunjukkan jitter sehubungan dengan bandwidth yang diterapkan. Meskipun nilai jitter secara signifikan rendah untuk node yang dipertimbangkan untuk proyek ini, hal ini membantu untuk memvisualisasikan kemacetan di dalam Raspberry Pi pada kecepatan data yang tinggi. Perhatikan nilai jitter yang relatif tinggi pada

yang digunakan. Jitter yang diukur untuk Raspberry Pi 4 model B ini sangat rendah karena pemrosesan paket yang masuk dengan cepat. Namun, masih dibatasi oleh antarmuka Ethernet 100 Mb/s/detik melalui hub bersama USB 2.0.

TABEL II. HASIL KASUS UJI #3

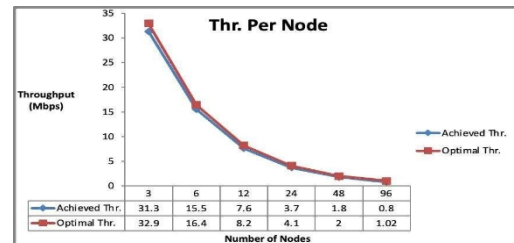
Kasus uji keempat adalah mengukur pembagian throughput berdasarkan jumlah klien. Iperf mampu meniru sejumlah klien hingga 192. Kasus pengujian ini paling sesuai dengan situasi di IoT di mana sejumlah besar perangkat mencoba menghubungkan dan mengirim data ke tujuan sebagai bagian dari proses pengumpulan data mereka dan perangkat gateway harus memenuhi persyaratan throughput dari semua koneksi secara bersamaan. Parameter multiple thread - P terkait dengan klien dan memungkinkan sisi klien untuk menjalankan beberapa thread secara bersamaan. Tentu saja, dengan menggunakan parameter ini, throughput akan dibagi ke dalam jumlah thread yang berjalan. Dengan fitur ini, kita dapat mensimulasikan sejumlah node yang mungkin secara aktif mencoba untuk terhubung dan mengirim data ke server.

Gbr. 11. Th. per node vs. jumlah node

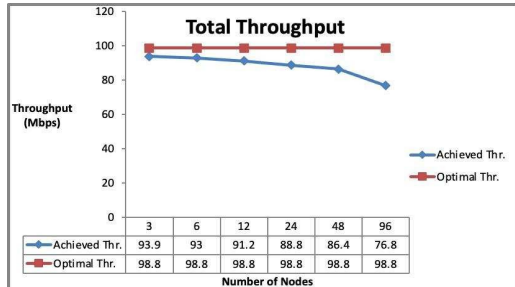
Seperti yang ditunjukkan pada Gambar 11 di atas, dapat dijelaskan bahwa untuk pengujian throughput pada 3 node yang melakukan request secara bersamaan dengan server, client mendapatkan throughput rata-rata sebesar 31.3 Mb/detik. Selain itu kita juga dapat melihat bahwa semakin sedikit node yang melakukan request ke server, maka throughput yang didapatkan tidak proporsional. Lebih lanjut menurut literatur, "aliran paralel dapat mencapai throughput yang lebih rendah dengan berperilaku sebagai satu aliran besar yang merupakan kombinasi dari n aliran dan mendapatkan bagian yang tidak adil dari bandwidth yang tersedia" [20]. Hasil yang diharapkan adalah pembagian throughput yang adil dibagi rata antara setiap node; namun, penelitian mereka juga memperkirakan bahwa akan ada beberapa penurunan kinerja karena banyak faktor yang terlibat dalam pengujian. Akhirnya, disimpulkan bahwa fungsi sakelar RPi dapat berjalan dengan baik tetapi mendapatkan hasil yang relatif tidak sama ketika jumlah node meningkat.

Dari Gambar 12, dapat dijelaskan bahwa untuk pengujian throughput klien yang berjumlah 3 hingga 96 klien, didapatkan hasil bahwa total throughput mengalami penurunan, mulai dari 96,3 Mbps hingga 2,21 Mbps. Hal ini dapat diartikan bahwa ketika semakin sedikit node yang melakukan request data ke server, maka total throughput semakin memenuhi bandwidth maksimum yang tersedia oleh sistem, yaitu sekitar 98,8 Mbps. Namun, ketika banyak node yang melakukan request data ke server seperti yang ditunjukkan pada kasus uji coba ini, penurunan throughput disebabkan oleh bandwidth yang tersedia oleh sistem sebagian digunakan untuk komunikasi

No	Fixed Throughput (Mbps/s)	Achieved Throughput (Mbps/s)	Jitter (ms)	Lost Packet / Total Packets	Packet Loss (%)
1.	10 Mb/s	10 Mb/s	1.68 ms	0/8504	0.00 %
2.	50 Mb/s	50 Mb/s	4.80 ms	25/42518	0.058 %
3.	100 Mb/s	95.8 Mb/s	13.56 ms	122/85034	0.143 %

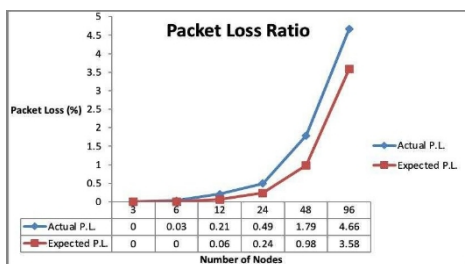


proses antara server dan klien. Karena banyaknya node yang berkomunikasi, akhirnya lalu lintas jaringan menjadi semakin padat. Pada pengujian ini nilai 96 adalah jumlah maksimum klien yang dapat ditangani oleh sistem. Dicoba untuk melakukan skenario dengan 100 hingga 192 klien, namun menghasilkan error dan sistem tidak dapat mengirimkan data. Dapat disimpulkan bahwa 96 node merupakan titik jenuh dari sistem.



Gbr. 12. Total throughput vs jumlah node

Kasus uji terakhir untuk mengukur rasio kehilangan paket berdasarkan jumlah klien untuk paket UDP. Iperf memberikan kita kemampuan untuk meniru jumlah klien serta paket UDP dengan menggabungkan parameter tertentu. Packet loss bertujuan untuk mengetahui berapa persen paket yang hilang karena keterbatasan perangkat keras atau sistem.



Gbr. 13. Rasio kehilangan paket vs. jumlah node

Pada gambar 13 di atas, dapat dijelaskan bahwa untuk hasil pengujian packet loss telah didapatkan hasil untuk pengaksesan jumlah Node mulai dari 3 hingga 96 node. Pada pengujian dengan 3 node yang melakukan request ke server, node mendapatkan rata-rata packet loss sebesar 0%, yang artinya semua data dapat terkirim 100%.

VII. KESIMPULAN

Kasus uji pertama dilakukan dengan menggunakan perintah ping yang sudah tersedia, yang merupakan alat yang sangat umum. Mayoritas kasus dilakukan dengan Iperf, sebuah metode pengukuran aktif, yang sangat umum di kalangan administrator jaringan untuk membandingkan jaringan mereka. Setiap kasus pengujian memiliki kriteria pengujian yang berbeda dan parameter QoS yang berbeda dievaluasi. Hasil pengujian dijelaskan secara rinci dalam bentuk tabel dan grafik. Tabel digunakan untuk keakuratan hasil dan grafik digunakan untuk tujuan visualisasi. Hasilnya, keseluruhan persyaratan aplikasi Internet untuk parameter QoS diidentifikasi untuk memvalidasi hasil dari Raspberry Pi yang bersangkutan. Software Defined Networking adalah bidang penelitian yang sangat menarik dengan kemungkinan tak terbatas. Penggabungan IoT dengan bidang penelitian yang begitu kuat dapat menghasilkan inovasi yang signifikan terhadap hal-hal sehari-hari yang sudah tersedia. Sistem saat ini hanya menggunakan Open vSwitch

sebagai sakelar virtual. Namun demikian, masih banyak lagi yang tersedia yang mungkin sesuai untuk skenario yang berbeda. Hal yang sama berlaku untuk pilihan pengontrol. Selain itu, dengan kemampuan Software Defined Networking, Raspberry Pi dapat dikonversi menjadi router, firewall, atau bahkan perangkat penyeimbang beban, semuanya tergantung pada kemampuan pemrograman sakelar virtual yang digunakan.

REFERENSI

- [1] Rechia, F. S., Syrotiuk, V. R., Ahn, G.-J., & Huang, D. (2016). Evaluasi Teknik Virtualisasi Jaringan Berbasis SDN. Diambil kembali dari https://repository.asu.edu/attachments/170581/content/StallRechia_asu_0010N_15909.pdf
- [2] Cisco. (2019). Mendefinisikan Masa Depan Internet Tentang Cisco. Diambil dari www.cisco.com.
- [3] Wang, S., Chavez, K. G., Kandeepan, S., & Zanna, P. (2018). Testbed jaringan berbasis perangkat lunak terkecil di dunia: Kinerja dan keamanan. Dalam Simposium Operasi dan Manajemen Jaringan IEEE/IFIP: Manajemen Kognitif di Dunia Maya, NOMS 2018.
- [4] Persatuan Insinyur Listrik dan Elektronika. (2016). Konferensi IEEE 2015 tentang Virtualisasi Fungsi Jaringan dan Jaringan yang Ditentukan Perangkat Lunak, NFV-SDN 2015. Konferensi IEEE 2015 tentang Virtualisasi Fungsi Jaringan dan Jaringan yang Ditentukan Perangkat Lunak, NFV-SDN 2015.
- [5] Chu, T. W., Shen, C. A., & Wu, C. W. (2018). Desain bersama perangkat keras dan perangkat lunak dari QoS yang dapat dikonfigurasi untuk streaming video berdasarkan protokol OpenFlow dan platform NetFPGA. *Multimedia Tools and Applications*, 77(7), 9071-9091.
- [6] Huang, H., Zhu, J., & Zhang, L. (2014). Kerangka kerja manajemen berbasis SDN untuk perangkat IoT. Dalam Publikasi Konferensi IET (Vol. 2014). <https://doi.org/10.1049/cp.2014.0680>
- [7] Y. Ma dkk., "Pengembangan dan implementasi kasus uji SDN," Konferensi Internasional ke-17 tentang Teknologi Komunikasi Lanjutan (ICACT) 2015, Seoul, 2015, hlm. 618-621.
- [8] Feamster, N., Rexford, J., & Zegura, E. (2014). Jalan menuju SDN: Sejarah intelektual jaringan yang dapat diprogram. Dalam *Computer Communication Review* (Vol. 44).
- [9] T. Luo dan S. Yu, "Mekanisme Kontrol dan Komunikasi SoftRouter," COIN-NGNCON 2006 - Konferensi Internasional Gabungan tentang Internet Optik dan Jaringan Generasi Berikutnya, Jeju, 2006, hal. 109-111.
- [10] Y. Jimenez, JA Cordero dan C. Cervelló-Pastor, "Mengukur ketahanan lapisan kontrol SDN," Simposium Internasional IFIP / IEEE 2015 tentang Manajemen Jaringan Terpadu (IM), Ottawa, ON, 2015, hlm. 774-777.
- [11] Tayyaba, S. K., Shah, M. A., Khan, O. A., & Ahmed, A. W. (2017). Internet of things (IoT) berbasis jaringan yang ditentukan perangkat lunak (SDN): Sebuah jalan di depan. *ACM International Conference Proceeding Series*, Part F130522. <https://doi.org/10.1145/3102304.3102319>
- [12] Yang, Y. Y., Yang, C. T., Chen, S. T., Cheng, W. H., & Jiang, F. C. (2015). Implementasi sistem pemantau lalu lintas jaringan dengan SDN. *Prosiding - Konferensi Perangkat Lunak dan Aplikasi Komputer Internasional*, 3, 631-634.
- [13] H. Kim, J. Kim dan Y. Ko, "Mengembangkan testbed OpenFlow yang hemat biaya untuk Software Defined Networking skala kecil," 16th International Conference on [14] .-L. Tseng dan F. J. Lin, "Memperluas skalabilitas platform IoT / M2M dengan komputasi Kabut," dalam 2018 IEEE 4th World Forum on Internet of Things (WF-IoT), 2018, Hal. 825-830.
- [14] Ohira, Kenji. "Evaluasi kinerja switch mirroring berbasis OpenFlow pada laptop/Raspberry Pi." *Prosiding Konferensi Internasional Kesembilan tentang Teknologi Internet Masa Depan*. ACM, 2014.
- [15] C. Lin, T. Hu dan H. Chan, "Implementasi pengiriman multi-path untuk aliran data menggunakan papan Raspberry Pi di jaringan yang ditentukan oleh perangkat lunak," 2017 IEEE 8th International Conference on Awareness Science and Technology (ICAST), Taichung, 2017, hlm. 330-333.
- [16] M. Priyadarsini, P. Bera dan R. Bampal, "Analisis kinerja arsitektur pengendali jaringan yang didefinisikan perangkat lunak-sebuah survei berbasis simulasi," Konferensi Internasional Komunikasi Nirkabel, Pemrosesan Sinyal, dan Jaringan (WiSPNET) 2017, Chennai, 2017, pp. 1929-1935.
- [17] C. Fancy dan M. Pushpalatha, "Evaluasi kinerja pengendali SDN POX dan lampu sorot di lingkungan emulasi mininet," Konferensi Internasional tentang Sistem Berkelanjutan Cerdas (ICISS) 2017, Palladam, 2017, hlm. 695-699.
- [18] R. G. Barba, M. Criollo, N. Aimacaña, C. Manosalvas dan C. Silva-Cardenas, "Kebijakan QoS untuk Meningkatkan Performa di Kampus Akademik dan Jaringan SDN," Konferensi Komunikasi Amerika Latin IEEE ke-10 (LATINCOM) 2018, Guadalajara, 2018, hlm. 1-6.
- [19] S. Han dan S. Lee, "Mengimplementasikan SDN dan jaringan yang dapat diprogram berbasis jaringan-hypervisor menggunakan Pi stack switch," Konferensi Internasional Konvergensi Teknologi Informasi dan Komunikasi 2015 (ICTC), Jeju, 2015, hlm. 579-581.
- [20] A. Malishevskiy, D. Gurkan, L. Dane, R. Narisetty, S. Narayan dan S. Bailey, "Manajemen Jaringan Berbasis OpenFlow dengan Visualisasi Elemen yang Dikelola," 2014 Third GENI Research and Educational

Experiment Workshop, Atlanta, GA, 2014, hal. 73-74.